TA Recruitment System
Complete Technical & User Manual

End-to-End Workflows, Architecture Guidelines, and Implementation Specifications

Development Team: BUPT Software Engineering Project Group

Team Members (GitHub): gdt1205, Gaojie13, Ddark-123, MasterYibo, Kerui-aua, wangyanzhou

Tech Stack: Java Servlet 4.0 + JSP 2.3 + Tomcat 7 + JSON Persistence

Document Version: v1.0.0 (Comprehensive Edition - v3)

Release Date: May 2026

1. System Overview & Vision

1.1 Project Background and Problem Resolution

In the higher education system, the recruitment and management of Teaching Assistants (TAs) is often a cumbersome process. Module Organizers (MO) struggle to efficiently screen candidates with the right academic backgrounds, while students (TAs) lack a transparent, real-time channel to understand job requirements and track their application progress. Furthermore, administrators (ADMIN) need to strictly prevent academic overload (e.g., TAs taking on too many courses simultaneously) and audit core system logs.

The "TA Recruitment System" provides an end-to-end digital solution. It covers the entire lifecycle from MOs publishing vacancies, TAs browsing and submitting CVs online, MOs reviewing and making interview decisions, to ADMINs executing global compliance checks and exporting data. This transforms the recruitment process from fragmented communication into an efficient, standardized, and closed-loop system.

1.2 Technology Stack & Database-Free Persistence Design

To ensure the system remains lightweight, easy to deploy, and highly dynamic, this project intentionally avoids traditional relational databases (like MySQL) or NoSQL databases. Instead, it utilizes a pure file-driven data persistence architecture:

- **Backend Core:** Java 11 / Java Servlet 4.0, responsible for receiving HTTP requests, executing core business validation, and orchestrating service workflows.
- **Frontend View:** JSP 2.3 + Native JavaScript & CSS. We strictly adhere to a traditional multi-page rendering architecture without complex frontend frameworks.
- **Data Serialization:** Google Gson library, handling two-way serialization between in-memory entity objects and on-disk JSON files.
- **Persistence Medium:** Local structured JSON file system. All system states, user profiles, and application data are saved under the server's `data/` directory.
- **Build & Container:** Maven Wrapper + Embedded Tomcat (`tomcat7-maven-plugin`) for one-click local compilation and startup.

2. Technical Architecture & Layering Conventions

2.1 Traditional Java Web Multi-Page Architecture

The project follows strict architectural layering. Developers are prohibited from making cross-layer calls or writing core business logic in the view layer. This convention is a strict safeguard to ensure parallel development across the team without code conflicts:

Layer Responsibilities (Mandatory Guidelines):

1. **controller/:** The primary entry point for HTTP requests. Its sole responsibility is to parse request parameters, call the corresponding `service` methods, and perform page redirection or forwarding based on the execution result. Complex `if/else` business rules are forbidden here.
2. **service/:** The core brain of the system. Handles all business rule validation, workload limit checks, state machine transitions, authorization verifications, and multi-module workflow orchestration.
3. **repository/:** Dedicated exclusively to reading and writing local JSON files. It is strictly forbidden to include any business-related code in this layer to ensure the purity and atomicity of data access.
4. **model/:** Defines basic entity fields, attributes, and getter/setter methods only. No business behaviors are permitted.
5. **util/:** Contains underlying common components unrelated to specific business logic, such as time formatters, unique ID generators, and CSV export engines.

2.2 Project Directory Structure & File Mapping

The physical directory structure of the project is arranged as follows. Any new feature development must be archived according to this layout:

```
software-engineering/
├── src/
│   ├── main/
│   │   ├── java/com/bupt/tarecruit/
│   │   │   ├── controller/      # Servlet implementations
```

```
│   │   │   ├── model/           # Domain entities (User, Job, Application)
│   │   │   ├── repository/      # JSON file read/write operations
│   │   │   ├── service/         # Business logic components
│   │   │   ├── util/            # Utilities (Time, ID, CSV exports)
│   │   │   └── web/             # Auth Filters and web infrastructure
│   │   └── webapp/
│   │       ├── WEB-INF/
│   │       ├── jsp/             # JSP view files
│   │       ├── css/             # Shared stylesheets
│   │       └── js/              # Browser-side interaction scripts
data/
├── users/                       # User profiles (USER_<userId>.json)
├── jobs/                        # Published jobs (JOB_<jobId>.json)
├── applications/                # Submitted applications
(<appId>_application.json)
├── notifications/               # Notification messages (NOTI_<id>.json)
└── operation-logs/              # Audit logs (LOG_<id>.json)
```

3. Environment Setup & Quick Start Guide

3.1 Prerequisites

Since the project includes the Maven Wrapper, developers only need to meet the following minimum requirements on their local machines:

- Java Development Kit (JDK) 11 or higher, with the `JAVA_HOME` environment variable configured.
- Git terminal tools (for code version management and synchronization).
- No separate Maven installation is required. The system will automatically download and lock the correct Maven version via the built-in `mvnw` command.

3.2 Starting the Service & Port Configuration

In the project root directory, open a terminal (PowerShell or Prompt) and run the following command to download dependencies, compile, and start the embedded Tomcat server:

```
.\mvnw.cmd tomcat7:run
```

Once the server starts successfully, the default listening port is `8080`. You can access the system homepage by navigating to:

`http://localhost:8080/`

If port `8080` is already occupied on your local machine, you can dynamically change the listening port (e.g., to `8081`) by passing a Java property parameter at runtime:

```
.\mvnw.cmd -Dmaven.tomcat.port=8081 tomcat7:run
```

3.3 Default Seed Accounts

The system's `data/users/` directory comes with built-in seed test data. You can use the following accounts directly for development and demonstration:

System Role	User ID	Password	Account Status & Permissions
ADMIN	`ADMIN001`	`password123`	System Administrator. Has permissions for account freezing, global monitoring, log reviewing, and CSV data exports.
MO	`MO001`	`password123`	Module Organizer. Has permissions to publish, edit, and close jobs, as well as review and make decisions on applicants.
TA	`TA001`	`password123`	Seeded TA account with pre-filled basic profile data, a default CV path, and historical application records.
TA	`TA002`	`123456`	A clean, new TA account. Useful for testing data entry and job applications from scratch.

4. Global Core Business Rules & Security Measures

4.1 Role-Based Access Control (RBAC) & Filter Design

The security architecture of this system is centrally controlled by the `AuthFilter.java`. This filter intercepts every HTTP request before it reaches the Servlets and executes three layers of validation:

- **Identity Verification:** Checks if an authenticated user object exists in the Session. If not, the request is safely redirected to the public login path `/login`.
- **Path Permission Matching:** The system assigns permissions based on URL prefixes. `/ta/*` routes require the `TA` role; `/mo/*` routes require the `MO` role; `/ad/*` routes require the `ADMIN` role. Unauthorized access will return an HTTP 403 error or force a redirect.
- **Dynamic Risk Control Interception:** Even with a valid Session token, the Filter reads the corresponding JSON file status in real-time. If the account has been marked as `Frozen` by an administrator, all operational access is immediately cut off at the authentication gateway.

4.2 Entity Identifiers (ID) & State Machine Standards

To ensure uniqueness and naming consistency of data files during parallel development, precise engineering rules dictate ID generation and state enums for core entities:

Entity Object	ID Naming Convention	Status Enums	Storage Strategy
User	Prefix + Auto-increment (e.g., `TA001`)	`Active` / `Frozen`	`USER_<userId>.json`
Job	`JOB_` + Random UUID / Timestamp	`Open` / `Closed`	`JOB_<jobId>.json`
Application	`APP_` + Random UUID	`Pending` / `Interview` / `Approved` / `Rejected`	`<applicationId>_appli`
Notification	`NOTI_` + Timestamp sequence	`Unread` / `Read`	`NOTI_<id>.json`

5. Teaching Assistant (TA) Module & Workflows

5.1 [TA001] Personal Profile & Academic Resume Management

Upon logging in for the first time, students must navigate to the personal center to complete their basic profile. The server-side service layer enforces strict non-null and format validations. All fields are mandatory; incomplete profiles will result in the server rejecting all subsequent job application requests.

Editable attributes include: Major, Current Grade, Completed Core Courses, GPA/Comprehensive Score, Relevant Project Experience, and CET6 Score (used as a hard filter for specific foreign or high-difficulty courses).

5.2 [TA002] CV File Upload Architecture

The system provides a dedicated CV upload dashboard. TAs can upload their personal resume in PDF format. After the backend Servlet receives the multipart form data, it physically writes the file into the server's `data/uploads/` directory, renaming it to a unique file mapped to the user ID. The system only stores the relative path of this file in the user's JSON configuration. During MO reviews, the system uses the `CvFileServlet` to perform authorization checks and securely output the file stream.

5.3 [TA003 & TA007-009] Job Board & Multi-Dimensional Search

TAs access the Job Board via the `/ta/jobs` route. This module only loads valid vacancies that have an `Open` status and have not passed their deadline. To help TAs quickly locate desired courses, the backend implements three advanced filtering mechanisms:

- **Fuzzy Search by Course Name:** Supports keyword filtering (e.g., "Data", "Network").
- **Precise Filtering by MO:** Allows categorizing jobs by the publishing professor (e.g., "Dr. Smith").
- **Dynamic Sorting by Time:** Supports ascending/descending sorting based on the job posting date or application deadline.

5.4 [TA005] Job Application & Personal Statement

When a TA applies for a job, clicking "Apply" triggers a chain of validations. First, the system checks for any unprocessed historical applications from this student for this specific job to prevent duplicate

submissions. Next, it calls the `AdminService` to verify if the student has hit the "3-course maximum" threshold. During submission, a text area is provided for students to optionally write a "Statement" (up to 500 words) highlighting their unique qualifications for the course.

5.5 [TA004 & TA006] Application Tracking & Notification Center

After applying, TAs can instantly track their progress under the "Application History" list at `/ta/home`. Every state machine change (e.g., from Pending to Interview or Approved) is reflected in real-time. Simultaneously, the system generates a notification message. When the TA logs in, the navigation bar highlights unread messages. By navigating to `/ta/notifications`, they can read the specific status changes and direct feedback from the instructor, with the ability to mark messages as read.

[UI Placeholder: TA Personal Home, Profile Management, and Application History View]

6. Module Organizer (MO) Module & Review Decisions

6.1 [MO001] Job Publishing Engine

After logging in, MOs can open a new vacancy form via the `/mo/publish` route. The publishing engine requires a complete, structured job description, including: Course Name, Job Type (Lab TA, Grading TA, Q&A TA), Hard Requirements, Expected Weekly Workload (Hours/Week), and an exact Application Deadline (YYYY-MM-DD HH:mm:ss). Upon submission, the system generates a unique `JOB_ID` and writes it to disk.

6.2 [MO004] Job Lifecycle & Automatic Closure Mechanism

Published jobs are not immutable. MOs can perform secondary edits (e.g., modifying requirements) at any time via the `/mo/positions` page. If a job secures enough candidates, the MO can manually execute a "Down" (Take Down) operation. Furthermore, the query logic includes a time comparison: once the current server time exceeds the job's set deadline, the job is automatically classified as `Closed` on the frontend. It will no longer accept new applications, but previously submitted applications are retained for the MO to continue reviewing.

6.3 [MO002 & MO003] Applicant Review & Status Transitions

Upon receiving applications, MOs enter the "Applicants List". In this dedicated workspace, MOs can comprehensively compare candidates:

- **Background Check:** View the TA's academic history, GPA, CET6 score, and personal statement with one click.
- **CV Review:** By clicking "View CV", the system securely invokes the underlying file stream to render and display the TA's uploaded PDF resume directly in the browser.
- **Decision & Feedback Dispatch:** MOs can execute status changes using three clear branching paths: `Interview`, `Approved`, or `Rejected`. Concurrently, the MO must provide written feedback (e.g., interview scheduling details or reasons for rejection) in the text box. Once saved, the system automatically updates the Application JSON file and pushes a real-time notification to the student.

[UI Placeholder: MO Job Management, Applicant List Review, and Decision View]

7. System Administrator (Admin) Global Control & Auditing

7.1 [AD001] 3-Course Limit Hard Interception (Core Business Rule)

To ensure TAs do not compromise their primary academic and research duties, the system enforces a strict global quota: "A single TA can take on a maximum of 3 TA courses." This rule is defended by the `AdminService` at the highest business level. When a TA clicks "Apply," the system deeply scans the `data/applications/` directory, counting all applications in the `Approved` state for that TA. If the count reaches 3, the backend immediately halts the operation and throws a compliance warning. The Admin dashboard provides a global highlighted view to monitor students operating at maximum capacity.

7.2 [AD002] Recruitment Progress Monitoring & Global Alerts

Administrators have a dynamic global perspective via the `/ad/projects` dashboard to monitor recruitment progress across all disciplines. This warning board clearly highlights jobs with "Zero Applications" or those that are "Published for a long time but unfilled." For jobs nearing the start date with a severe lack of TAs, the board flags them with yellow or red alerts. ADMINs can click a button to send an automated system reminder to the respective MO, preventing potential teaching disruptions.

7.3 [AD005] Risk Control & Account Freezing

If the system detects abnormal operations, malicious spamming, or fraudulent document submissions from any account (TA or MO), the ADMIN has the supreme authority to enter the `/ad/accounts` page. Administrators can locate users by ID, Name, or Department and click "Freeze Account". This action instantly overwrites the `status` field in the corresponding `USER_*.json` file to `Frozen`. Once frozen, the user's Session privileges are entirely revoked at the global filter level. They are blocked from logging in, publishing, applying, or reviewing until the administrator manually executes an "Unfreeze" action.

7.4 [AD003 & AD004] Full Data CSV Export & Audit Logs

To meet the needs of administrative reporting, offline archiving, or future data migration, the ADMIN module encapsulates powerful data tools:

- **[AD003] Full TA Data Export:** Admins can invoke the `CsvExportUtil` with one click. The system traverses all TA profiles and historical application records, mapping and flattening the fields. It exports core metrics (Student ID, Name, Research Area, Total Approved Courses) into a standard CSV report, saved in the server's `exports/` directory.
- **[AD004] System Audit Trail:** To ensure system traceability, the underlying architecture features global operation listeners. Whenever a user performs a core action involving status changes (e.g., logging in, publishing jobs, changing review statuses, freezing accounts), the system calls the log utility. It packages the User ID, action type, target, detailed parameters, and high-precision timestamp into a JSON object, appending it to the log files in `data/operation-logs/`. Administrators can scroll indefinitely through the `/ad/logs` page to ensure every sensitive action is documented.

[UI Placeholder: Admin Global Account Management, Risk Control, and Audit Log View]

8. System Routing Table (URLs) & Access Permissions

To provide the development team with a clear global reference during API integration and permission configuration, below is the complete list of supported HTTP paths, their corresponding Servlets, and mandatory role access permissions:

8.1 Public & Authentication Routes (Open Access)

URL Pattern	Backend Servlet Class	HTTP Method	Function / Responsibility
`/`	IndexServlet	GET	System root; guides non-logged-in users to the login page.
`/login`	LoginServlet.java	GET / POST	Displays login UI; validates credentials and sets Session.
`/logout`	LogoutServlet.java	GET	Clears the current user's Session and redirects to home.
`/register`	RegisterServlet.java	GET / POST	Provides online self-registration for new TA or MO accounts.

8.2 TA Routes (Restricted to TA Role)

URL Pattern	Core Function	HTTP Method	Data Persistence Target
`/ta/enter`	TA Workspace entry router	GET	No direct change; handles homepage redirection.
`/ta/home`	TA Personal Homepage	GET	Loads profile and application history.
`/ta/jobs`	Job Board (Browse & Search)	GET	Filters all `Open` job JSON files.
`/ta/profile`	Edit & save academic profile	POST	Overwrites `USER_<userId>.json`.
`/ta/uploadCv`	Handle PDF CV uploads	POST	Writes to `data/uploads/` & updates user profile.
`/ta/apply`	Submit job application	POST	Creates new file in `data/applications/`.
`/ta/notifications`	View feedback in Notification Center	GET	Updates unread status to `Read`.

8.3 MO Routes (Restricted to MO Role)

URL Pattern	Core Function	HTTP Method	Data Persistence Target
`/mo/home`	MO Dashboard Homepage	GET	Loads all courses and applicants linked to the MO.
`/mo/publish`	Publish new TA vacancies	GET / POST	Creates new `JOB_*.json` in `data/jobs/`.
`/mo/positions`	Manage published jobs list	GET	Reads all historical jobs posted by the MO.
`/mo/edit`	Edit existing job descriptions	POST	Updates `JOB_<jobId>.json`.
`/mo/job/down`	Manually take down/ close a job	POST	Updates Job status to `Closed`.
`/mo/job/applicants`	View all TAs applying for a course	GET	Cross-references Application and User JSON files.
`/mo/application/action`	Execute decisions (Interview/Approve/ Reject)	POST	Modifies Application status and creates new Notification.

8.4 Admin Routes (Restricted to ADMIN Role)

URL Pattern	Core Function	HTTP Method	Data Persistence Target
`/ad/home`	Admin Supreme Dashboard Homepage	GET	Loads global statistical snapshots.
`/ad/accounts`	Global User Account Management	GET	Retrieves states of all registered users.
`/ad/accounts/action`	Execute Risk Control (Freeze/Unfreeze)	POST	Updates target User status to `Frozen` or `Active`.
`/ad/projects`	Global Progress Monitoring & Alerts	GET	Cross-references all Job and Application states.
`/ad/logs`	Review Audit Trails (Audit Logs)	GET	Streams and paginates `data/operation-logs/`.

9. Test Automation & Standard Demo Script

9.1 JUnit 4 Automated Test Coverage

This project places a high priority on software reliability. Currently, the system features a comprehensive automated test suite covering the Controller, Service, and Repository layers. By running the following command in the project root, you can execute all tests with one click:

```
.\mvnw.cmd test
```

The test suite currently includes **70 core passing tests** on the main branch. Key testing scenarios include:

- **Authentication Tests (Login/Logout/RegisterServletTest):** Covers correct password logins, incorrect password rejections, empty field validations, and duplicate username blocks.
- **Business Rule Tests (Admin/ApplicationServiceTest):** Focuses on boundary testing for the "3-course limit". Verifies that a TA with 2 approved courses can apply, but a 4th application attempt (when holding 3 approved) is accurately intercepted by the system, throwing the expected exception.
- **Persistence Reliability Tests (Repository Test):** Simulates stream reading/writing of local JSON files under high concurrency to ensure no field loss or deadlocks occur during Gson serialization.
- **Acceptance Integration Tests (TaModuleTestReportTest):** Simulates the complete end-to-end black-box business chain from user login and profile updates to the final report export.

9.2 Six-Step Closed-Loop Standard Demo Script

When presenting the system to evaluation experts or academic administrators, we strongly recommend following this "Golden Closed-Loop" sequence. This flow perfectly demonstrates the interactions between all three roles and the underlying rules within a single demo run:

1. **Step 1: MO Publishes a Job**
   Log in as the Module Organizer `MO001` (password: `password123`). Navigate to the publish page and create a new TA position (e.g., "EBU5476 Data Structures"). Set high requirements and a 20-hour weekly workload, then click Publish.
2. **Step 2: TA Browses and Updates Profile**
   Log out of the MO account and log in as the Teaching Assistant `TA001` (password: `password123`).

Go to the profile page, update the GPA and CET6 scores, and upload a standard PDF resume. Then, enter the Job Board and use a keyword search ("Data Structures") to find the job the MO just published.

3. **Step 3: TA Submits an Application**
   On the job details page, write a statement of strengths in the Statement box and click "Apply". Next, show the audience the `/ta/home` page to verify that the application's current status is `Pending`.
4. **Step 4: MO Reviews, Checks CV, and Decides**
   Switch back to the instructor account `MO001`. Go to the dashboard and open the applicant list for the job. You will see `TA001`'s application. Click "View CV" to demonstrate how the system securely streams the student's PDF resume in the browser. Then, click to update the status to `Interview`, add interview details in the feedback box, and save.
5. **Step 5: TA Receives Notification**
   Switch back to the student account `TA001`. Draw the audience's attention to the notification highlight on the top navigation bar. Enter the Notification Center (`/ta/notifications`) to show the interview arrangement and direct feedback from the teacher, then click "Mark as Read".
6. **Step 6: ADMIN Executes Audit and Risk Control**
   Finally, log in as the top-level Administrator `ADMIN001` (password: `password123`). First, check the `/ad/projects` dashboard to view macro statistics and alerts. Then, go to the `/ad/logs` page to show the audience the high-precision Audit Trail generated by the previous five steps, proving the system's exceptional traceability and security. This concludes the perfect closed-loop demonstration.